

DiabetesRisk AI

Five detailed answers (dissertation / project rationale)

Grounded in repository: README.md, ml/train_model.py, ml/api.py, ml/models/model_meta.json, diabetes_app (Rails 7).

Note: .docx dissertation files in /file are not machine-read here; this document reflects the implemented codebase and README.

1. Why I chose this project

Diabetes mellitus is a major global public-health burden. Early identification of elevated risk can motivate lifestyle change and timely clinical follow-up (e.g. fasting glucose, HbA1c), which aligns with preventive medicine and health informatics.

From a computing research angle, the project sits at a sweet spot: (a) a well-documented, reproducible benchmark dataset (Pima Indians Diabetes, UCI id 34) cited in decades of ML literature; (b) a clear binary classification target (Outcome 0/1); (c) realistic data issues (missing values encoded as zeros, class imbalance) that force serious preprocessing and evaluation discipline rather than toy curve-fitting.

The implementation scope matches a dissertation-style systems contribution: a Python ML pipeline (scikit-learn, XGBoost, optional LightGBM, imbalanced-learn SMOTE, SHAP), a Flask REST API for inference and explainability, and a Ruby on Rails 7 web front end with validation, UX, and explicit clinical disclaimers. That end-to-end stack demonstrates integration skills (HTTP APIs, containers, environment configuration) that go beyond a notebook-only experiment.

Finally, the README frames explicit quantitative success gates (AUC-ROC ≥ 0.85 , F1 ≥ 0.80 , Recall ≥ 0.78), which makes the work auditable: success is defined before training, and the training script logs grid-search results and saves model_meta.json for traceability.

Pedagogically, the project forces engagement with the whole ML lifecycle: data acquisition (ucimlrepo vs CSV fallbacks), leakage-aware thinking (SMOTE on training design vs holdout evaluation in _honest_eval), hyperparameter search, serialisation (joblib), serving (Flask), and responsible UI copy (tooltips mirroring clinical definitions in PredictionsController, repeated warnings in layouts and result pages).

Social impact narrative: type 2 diabetes risk rises with obesity, sedentary behaviour, and family history; a transparent predictor plus SHAP explanations can support patient education in a supervised demo setting, provided the population limits of the Pima cohort are respected.

2. What is the future of this project

Short term (research / coursework): extend evaluation with calibration plots, cost-sensitive thresholds tuned for screening (prioritising recall), confusion matrices on the honest stratified holdout split already computed in train_model.py, and ablation studies (with vs without SMOTE, alternative imputations). The placeholder ml/generate_evidence.py points to integrating charts into the training run if formal evidence packs are required.

Medium term (software): harden the API (rate limits, authentication, structured logging, model version headers), add automated tests for the Rails controller and Flask endpoints, and CI that rebuilds Docker images (Dockerfile.ml, Dockerfile.rails, docker-compose.yml). The repo already documents Docker Hub publishing via bin/push-dockerhub.sh and docker-compose.hub.yml for reproducible deployment.

Long term (clinical translation): the README disclaimer is explicit: the tool is not clinically validated and must not drive patient decisions. A credible path forward would be external validation on new cohorts (different ethnicity, geography, age/sex mix), prospective study design, regulatory input where applicable, and integration with electronic health records or FHIR-based workflows rather than manual numeric forms only.

Explainability already exists (SHAP in `api.py` with `TreeExplainer` fallback to a generic `Explainer`). Future work could add cohort-specific fairness audits, uncertainty quantification, and human-in-the-loop review interfaces for clinicians.

Data futures: augment or replace the Pima table with multi-ethnic longitudinal EHR extracts, time-series glucose traces (CGM), or prescription and comorbidity codes; each shift demands new feature engineering, missing-data policies, and re-tuned class weights because prevalence and label definitions change.

Product futures: mobile-friendly forms, localisation, accessibility (WCAG), audit logs of predictions for governance, and model cards linked from the About page could turn the prototype into a teachable reference architecture for digital-health startups or hospital IT.

3. Why we use this model (algorithm choice)

The project does not assume a single algorithm upfront. `train_model.py` runs `GridSearchCV` with 5-fold stratified cross-validation, optimising `roc_auc` for five families: Logistic Regression, Random Forest, XGBoost, LightGBM (if installed), and RBF SVM with `probability=True`. After CV, the best estimator by resampled-set AUC is selected, then metrics are also reported on SMOTE-resampled training space; importantly, `_honest_eval` computes holdout metrics on a stratified 80/20 split of the original (pre-SMOTE) labels using scaled features, which better reflects generalisation behaviour than training-only scores on synthetic minority points alone.

The currently saved artefact `ml/models/model_meta.json` names `RandomForest` as `model_name` with `best_params` `n_estimators=200` and `max_depth=null` (fully grown trees subject to `min_samples` constraints). Random forests are a strong default for tabular clinical-style data: they capture non-linear interactions, are relatively robust to scaling after `StandardScaler`, tolerate noisy features better than a single deep tree, and work well with `TreeExplainer` for SHAP in `api.py`.

If resampled metrics fall below the dissertation gates, the pipeline can load `best_model_pretrained.pkl` or a remote joblib URL (`PRETRAINED_MODEL_URL`), then fall back to the best grid-search model. That design acknowledges that small public datasets can produce unstable headline metrics and keeps a documented recovery path.

Hyperparameter grids (abbreviated): LogisticRegression over C in {0.1,1,10} with `lbfgs`; RandomForest over `n_estimators` {100,200} and `max_depth` {None,8,12}; XGBoost over `n_estimators`, `max_depth`, `learning_rate`; optional LightGBM with analogous ranges; SVM RBF over C and gamma. `GridSearchCV` uses `n_jobs=-1` for throughput and refits the best `roc_auc` estimator per family.

Why not a single deep neural net here: with only eight numeric inputs and hundreds of rows, tree ensembles and regularised linear models are competitive, easier to tune, faster to train on CPU, and align with `TreeExplainer` for low-latency per-request explanations in the API.

4. What dataset we train this model on

Primary dataset: Pima Indians Diabetes Database from the UCI Machine Learning Repository (dataset id 34). The training script prefers `ucimlrepo.fetch_ucirepo(id=34)`; if that fails, it falls back to public CSV mirrors (Plotly datasets, a GitHub gist), so training remains reproducible offline subject to network availability for fallbacks.

Rows and columns: README and Rails about page describe 768 records and 8 input features (Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age) plus binary target Outcome. Population caveat (also in README and UI disclaimers): the cohort is adult females of Pima Indian heritage aged 21+; any deployment narrative must warn against extrapolation to other populations without re-training and validation.

Preprocessing in preprocess(): biologically invalid zeros in Glucose, BloodPressure, SkinThickness, Insulin, and BMI are treated as missing, replaced by column medians; features are StandardScaler-normalised; SMOTE balances classes to mitigate the roughly 65/35 imbalance noted in the About page. These choices directly affect learned decision boundaries and must be disclosed when interpreting metrics.

Citation (from README): Smith, J.W., Everhart, J.E., Dickson, W.C., Knowler, W.C., & Johannes, R.S. (1988). Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. Proceedings of the Annual Symposium on Computer Application in Medical Care, 261-265.

Feature semantics (aligned with Rails tooltips): Glucose is plasma glucose 2 hours after OGTT; BloodPressure is diastolic mm Hg; SkinThickness is triceps skin fold mm; Insulin is 2-hour serum insulin; BMI is weight/height²; DiabetesPedigreeFunction encodes genetic risk; Pregnancies and Age are counts/years. Training replaces sentinel zeros only where clinically implausible, matching domain knowledge rather than blind zero imputation.

Class distribution: train_model.py logs value_counts on Outcome after load; the About page states an approximate 65/35 imbalance motivating SMOTE. Always report both raw prevalence and post-SMOTE training sizes when writing methodology chapters.

5. What is the outcome of this project

Trained model artefacts: ml/models/best_model.pkl, scaler.pkl, and model_meta.json. According to the checked-in model_meta.json snapshot, the selected classifier is RandomForest; resampled training-space metrics are reported as auc=1.0, f1=1.0, recall=1.0 (these are optimistic because they include SMOTE-generated synthetic examples), while honest holdout metrics are more conservative: holdout_auc about 0.9998, holdout_f1 about 0.9908, holdout_recall 1.0, holdout_accuracy about 0.9935, holdout_precision about 0.9818. Readers should treat holdout metrics as the primary success story for generalisation claims.

Software outcomes: Flask service (api.py) exposes /health, /model_info, and POST /predict returning label, probabilities, SHAP dictionary, plain-language guidance, and a legal-style disclaimer. Rails PredictionsController posts JSON to the API, renders result.html.erb with Chart.js SHAP bar chart, and exposes /about and /health for transparency.

Operations outcomes: multi-stage Dockerfiles and compose files enable local and hub-based deployment; environment variables ML_API_URL and ML_API_PORT wire the stack. The project delivers a research-grade decision-support prototype suitable for dissertation demonstration, with clear boundaries on clinical use.

User-visible outcomes: the Rails UI collects eight validated numeric fields (min/max from FEATURE_RANGES), calls POST /predict, shows confidence bars, probability split, SHAP bar chart (Chart.js), clinician-style guidance strings from api.py (_guidance), and surfaces model_name plus auc on the results card for auditability.

Research integrity note: resampled-set AUC/F1/recall in model_meta can reach 1.0 because the model is scored on SMOTE-balanced data it was trained on; the holdout_* block is the fairer narrative for external claims. Discuss limitations: tiny test fold, possible overlap between similar patients, and lack of external cohort

validation.

Dependencies snapshot (ml/requirements.txt): pandas 2.1.4, numpy 1.26.4, scikit-learn 1.4.2, xgboost 2.0.3, lightgbm 4.3.0, imbalanced-learn 0.12.3, shap 0.44.1, flask 3.0.3, joblib 1.4.2, ucimlrepo 0.0.7. Rails uses Ruby 3.3.4 and Rails 7.1 per Gemfile for the web tier.